Análisis de Conflictos de Mesa Multi-Mesero — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar qué pasa cuando múltiples meseros atienden la misma mesa simultáneamente, cómo evitar solapamiento de pedidos, y qué mecanismos de protección se necesitan.

---

1. El problema real

Escenario típico
```
Mesa 5 (4 comensales)
  → Mesero A llega, toma pedido (2 hamburguesas, 2 cervezas)
  → Mesero A envía comanda → Cocina recibe → prepara
  → Mesero B llega (no sabe que A ya tomó pedido)
  → Mesero B toma pedido (1 postre, 1 café) → envía comanda
  → Cocina recibe SEGUNDA comanda para Mesa 5
  → Cocina prepara 2 hamburguesas + 2 cervezas + 1 postre + 1 café
  → Caja recibe 2 comandas separadas para Mesa 5
  → Al cobrar: ¿se cobra una o dos? ¿Se duplica?
```

Variantes del problema
1. **Solapamiento**: A y B toman pedidos simultáneamente → dos comandas separadas para la misma mesa
2. **Duplicación**: A toma pedido, B no lo ve, toma el mismo pedido → se duplica
3. **Conflicto de estado**: A marca mesa como "ocupada", B no lo ve, también marca como "ocupada"
4. **Conflicto de cobro**: A cobra parcial, B cobra parcial → doble cobro
5. **Conflicto de transferencia**: A transfiere mesa a B, pero B ya tomó pedido en otra mesa

---

2. Lo que el código actual hace

2.1 Estado de mesa (runtime)

El sistema mantiene `clientePorSlot` y `descuentosPorSlot` por mesa/llevar:

```javascript
// CrozzoPosMain.js:4265
let clientePorSlot = { mesa: {}, llevar: {} };
let descuentosPorSlot = { mesa: {}, llevar: {} };
```

Problema: Estos son locales al dispositivo. Si Tablet A marca Mesa 5 como ocupada, Tablet B NO lo sabe hasta que sync runtime se propague (2-4s de delay).

2.2 Comandas por mesa

Las comandas se almacenan en un array global `comandas[]`:

```javascript
// CrozzoPosMain.js:10072
comandas.unshift(comanda);
```

Problema: Cada dispositivo tiene su propia copia de `comandas[]`. Si Tablet A agrega una comanda, Tablet B no la ve hasta sync.

2.3 Sync runtime (mesas)

El runtime sync envía el estado de todas las mesas cada 2-4s:

```javascript
// CrozzoPosRuntimeCloud.js
crozzoPushPosRuntimeCloudNow() → pushMesaRows → crozzo_mesa_runtime
crozzoPullPosRuntimeCloud() → applyPosRuntimeSnapshot → crozzoHandleRemoteRuntimeUiSync
```

Problema: Delay de 2-4s. En ese tiempo, dos meseros pueden tomar pedidos en la misma mesa sin saberlo.

2.4 Deduplicación de comandas

El sistema usa `transaction_id` para evitar duplicados:

```javascript
// CrozzoComandasCloudSync.js:665
function findComandaForCloudPay(pay, row) {
  var tid = String(pay.transaction_id || (row && row.id) || '').trim();
  var byTid = list.find(function (c) {
    return c && c.transaction_id && String(c.transaction_id) === tid;
  });
  if (byTid) return byTid;  // ya existe, no duplicar
}
```

Esto evita que la MISMA comanda se duplique, pero NO evita que dos comandas DIFERENTES se creen para la misma mesa.

---

3. Lo que falta: detección de conflictos de mesa

3.1 Lo que NO existe

Mecanismo	Estado	Qué hace
Detección de mesa ocupada por otro mesero	❌ No existe	Avisar "Mesa 5 ya tiene pedido de Mesero A"
Bloqueo de mesa (lock)	❌ No existe	Impedir que B comande en Mesa 5 si A está activo
Merge de comandas por mesa	❌ No existe	Combinar comandas de A y B en una sola
Notificación de conflicto	❌ No existe	Avisar "Conflicto en Mesa 5: 2 meseros activos"
Resolución automática	❌ No existe	Decidir qué hacer cuando hay conflicto

3.2 Lo que SÍ existe (parcialmente)

Mecanismo	Estado	Limitación
`clientePorSlot`	✅ Existe	Solo local al dispositivo, no sincronizado en tiempo real
`descuentosPorSlot`	✅ Existe	Solo local al dispositivo
Sync runtime (mesas)	✅ Existe	Delay de 2-4s, no tiempo real
`transaction_id` dedup	✅ Existe	Evita duplicados de la MISMA comanda, no de comandas diferentes
`crozzoSlotCartMergePinned`	✅ Existe	Controla merge de carrito, pero no conflictos de mesa

---

4. Escenarios de conflicto y soluciones

4.1 Escenario: Dos meseros toman pedido en la misma mesa simultáneamente

```
T=0: Mesero A abre Mesa 5, empieza a tomar pedido
T=1: Mesero B abre Mesa 5 (no sabe que A está ahí)
T=2: A envía comanda (2 hamburguesas)
T=3: B envía comanda (1 postre)
T=4: Cocina recibe 2 comandas para Mesa 5
T=5: Caja recibe 2 comandas para Mesa 5
```

Problema: Cocina prepara 2 hamburguesas + 1 postre (OK, pero separados). Caja tiene 2 comandas separadas para la misma mesa. Al cobrar, ¿se cobra una o dos?

Solución propuesta:
1. **Detección**: Cuando B abre Mesa 5, el sistema verifica si hay comandas activas para esa mesa.
2. **Aviso**: "Mesa 5 tiene pedido de Mesero A (2 items). ¿Deseas agregar a esa comanda o crear una nueva?"
3. **Merge**: Si B elige "agregar", se combinan los items en la comanda existente.
4. **Separado**: Si B elige "nueva", se crea comanda separada pero vinculada a la mesa.

4.2 Escenario: Mesero B no ve que A ya tomó pedido

```
T=0: A toma pedido (2 hamburguesas) → envía comanda
T=1: Sync runtime se propaga (2-4s delay)
T=2: B abre Mesa 5 (sync aún no llegó) → no ve comanda de A
T=3: B toma pedido (2 hamburguesas) → envía comanda
T=4: Sync llega → B ve comanda de A
T=5: Ahora hay 2 comandas idénticas para Mesa 5
```

Problema: Duplicación de pedido. Cocina prepara 4 hamburguesas en lugar de 2.

Solución propuesta:
1. **Detección por items**: Si B intenta agregar items que ya están en la comanda de A, avisar "Estos items ya fueron pedidos por Mesero A".
2. **Detección por timestamp**: Si B abre Mesa 5 y hay comanda reciente (<5 min), avisar "Mesa 5 tiene pedido reciente de Mesero A".
3. **Merge automático**: Si los items son idénticos, no crear nueva comanda, solo notificar.

4.3 Escenario: Mesero A transfiere mesa a B

```
T=0: A toma pedido en Mesa 5 → envía comanda
T=1: A transfiere Mesa 5 a B
T=2: B recibe transferencia → abre Mesa 5
T=3: B toma pedido adicional → envía comanda
```

Problema: ¿La comanda de B se vincula a la de A o es independiente?

Solución propuesta:
1. **Transferencia explícita**: A transfiere la mesa → B recibe la comanda existente
2. **B puede agregar items** a la comanda existente
3. **B NO puede crear comanda nueva** sin cerrar la existente

4.4 Escenario: Mesa grande con múltiples meseros

```
Mesa 10 (12 comensales, 3 meseros atendiendo)
  → Mesero A toma pedido de grupo 1 (4 personas)
  → Mesero B toma pedido de grupo 2 (4 personas)
```

```
  → Mesero C toma pedido de grupo 3 (4 personas)
  → 3 comandas separadas para la misma mesa
```

Esto es LEGÍTIMO. Múltiples meseros pueden atender la misma mesa. El problema es cuando no se sabe que otro mesero ya tomó pedido.

Solución propuesta:
1. **Mesa compartida**: Permitir múltiples comandas por mesa
2. **Visualización**: Cada mesero ve las comandas existentes de la mesa
3. **Aviso**: "Mesa 10 tiene 2 comandas activas de Mesero A y B"
4. **Cobro unificado**: Al cobrar, se suman todas las comandas de la mesa

---

5. Mecanismos de protección necesarios

5.1 Detección de mesa ocupada (tiempo real)

```
Cuando un mesero abre una mesa:
  1. Consultar runtime sync: ¿hay comandas activas para esta mesa?
  2. Si sí → mostrar aviso: "Mesa X tiene pedido de Mesero Y (N items, hace M minutos)"
  3. Opciones:
     a) Ver pedido existente
     b) Agregar a pedido existente
     c) Crear comanda nueva (con confirmación)
```

Implementación:
- Usar `crozzoHandleRemoteRuntimeUiSync` para obtener estado actual de la mesa
- Verificar `comandas` array filtrado por `referencia === mesa`
- Mostrar modal de conflicto si hay comandas activas

5.2 Lock de mesa (soft lock)

```
Cuando un mesero empieza a tomar pedido en una mesa:
  1. Marcar mesa como "en edición por Mesero X" (runtime sync)
  2. Otros meseros ven: "Mesa X en edición por Mesero X"
  3. Si otro mesero intenta abrir → aviso: "Mesa X está siendo atendida por Mesero X"
  4. Lock se libera al enviar comanda o al cancelar
  5. Lock expira después de 5 minutos (por si el mesero se olvida)
```

Implementación:
- Agregar campo `editingBy` y `editingAt` al runtime de mesa
- Sync por runtime (2-4s delay)
- Expiración automática (5 min)

5.3 Merge de comandas por mesa

```
Cuando se detecta conflicto:
  1. Mostrar comandas existentes de la mesa
  2. Permitir agregar items a comanda existente
  3. O crear comanda nueva vinculada
  4. Al cobrar, sumar todas las comandas de la mesa
```

Implementación:

- Función `crozzoMergeComandaIntoExisting(mesa, newItems)`
- Vincular comandas por `mesaGroupKey`
- Sumar totales al cobrar

5.4 Notificación de conflicto (no invasiva)

```
Cuando hay conflicto:
  1. Banner pequeño en la parte superior: "⚠️ Mesa 5 tiene pedido de Mesero A"
  2. Sonido sutil (1 vez)
  3. El mesero puede ignorar o ver el pedido
  4. NO bloquea la pantalla (no modal invasivo)
  5. Se registra en auditoría
```

5.5 Resolución automática (para casos simples)

```
Si dos meseros agregan items DIFERENTES a la misma mesa:
  → Merge automático (sin preguntar)
  → Cocina recibe una sola comanda con todos los items
  → Caja cobra una sola cuenta

Si dos meseros agregan items IDÉNTICOS:
  → Aviso: "Estos items ya fueron pedidos"
  → Mesero decide si duplicar o no

Si un mesero intenta cobrar mientras otro está tomando pedido:
  → Bloquear cobro → "Mesa 5 tiene pedido en curso de Mesero A"
```

---

6. Prioridades de comunicación (orden de importancia)

Prioridad 1: Comanda → Cocina (CRÍTICO)
```
Objetivo: La cocina debe recibir el pedido en <2 segundos
Canal: WS push (tiempo real) + HTTP (fallback)
Verificación: Cocina imprime comanda automáticamente
Fallo: Si no llega → cocina no prepara → cliente espera
```

Prioridad 2: Mesa ocupada → Otros dispositivos (IMPORTANTE)
```
Objetivo: Los meseros deben saber qué mesas están ocupadas
Canal: Sync runtime (2-4s delay)
Verificación: Mesa muestra estado "ocupada" en todos los dispositivos
Fallo: Dos meseros toman pedido en la misma mesa → duplicación
```

Prioridad 3: Estado de comanda → Todos (IMPORTANTE)
```
Objetivo: Todos deben saber el estado de cada comanda
Canal: WS push + HTTP poll
Verificación: Mesero ve "listo" cuando cocina marca listo
Fallo: Mesero no sabe que el plato está listo → cliente espera
```

Prioridad 4: Inventario → Caja (SECUNDARIO)
```
```

```
Objetivo: Stock se descuenta al vender
Canal: Local + sync cloud
Verificación: Inventario refleja ventas
Fallo: Stock incorrecto, pero no afecta operación inmediata
```

Prioridad 5: Costeo → Reportes (SECUNDARIO)
```
Objetivo: Margen y rentabilidad se actualizan
Canal: Local + sync cloud
Verificación: Reportes reflejan ventas
Fallo: Reportes incorrectos, pero no afecta operación
```

---

7. Lo que falta implementar

7.1 Detección de conflictos de mesa

#	Componente	Qué hace	Prioridad	Esfuerzo
1	`crozzoMesaConflictCheck(mesa)`	Verifica si hay comandas activas en la mesa	🔴 Alta	1 día
2	`crozzoMesaLock(mesa, mesero)`	Marca mesa como "en edición por X"	🔴 Alta	1 día
3	`crozzoMesaUnlock(mesa)`	Libera lock al enviar/cancelar	🔴 Alta	0.5 día
4	Modal de conflicto (no invasivo)	"Mesa X tiene pedido de Mesero Y"	🔴 Alta	1 día
5	`crozzoMergeComandaIntoExisting(mesa, items)`	Agrega items a comanda existente	🟡 Media	2 días
6	Cobro unificado por mesa	Suma todas las comandas de la mesa	🟡 Media	1 día
7	Aviso de items duplicados	"Estos items ya fueron pedidos"	🟡 Media	1 día
8	Registro de auditoría de conflictos	Log de conflictos resueltos	🟢 Baja	0.5 día

7.2 Sync de ocupación de mesa (tiempo real)

#	Componente	Qué hace	Prioridad	Esfuerzo
1	`editingBy` en runtime de mesa	Quién está editando la mesa	🔴 Alta	0.5 día
2	`editingAt` en runtime de mesa	Cuándo empezó a editar	🔴 Alta	0.5 día
3	Expiración de lock (5 min)	Libera lock automáticamente	🔴 Alta	0.5 día
4	Sync de lock por runtime (2-4s)	Propaga lock a otros dispositivos	🔴 Alta	0.5 día
5	Visualización de lock en UI	"Mesa X en edición por Mesero Y"	🟡 Media	1 día

---

8. Criterios de éxito para conflictos de mesa

Criterio	Cómo verificar
Mesero B ve aviso al abrir mesa ocupada por A	Probar con 2 tablets
Mesero B puede agregar a comanda existente	Probar merge
Mesero B puede crear comanda nueva (con confirmación)	Probar con confirmación
Cocina no recibe duplicados	Verificar comandas en cocina
Caja cobra una sola cuenta por mesa	Verificar cobro unificado
Lock se libera al enviar/cancelar	Probar flujo completo
Lock expira después de 5 min	Probar timeout
Aviso no es invasivo (banner, no popup)	Probar en dispositivo real

---

9. Conclusión

Lo que ya funciona (no tocar)
- Comanda → Cocina (WS push + auto-print) ✅
- Deduplicación por transaction_id ✅
- Sync runtime (mesas) ✅
- Merge de carrito (LWW) ✅

Lo que falta (crítico)
- **Detección de mesa ocupada** (tiempo real)
- **Lock de mesa** (soft lock con expiración)
- **Merge de comandas** por mesa
- **Notificación de conflicto** (no invasiva)
- **Cobro unificado** por mesa

El objetivo final
> "Dos meseros pueden atender la misma mesa. El sistema detecta el conflicto, avisa sin ser invasivo, y permite merge o separación. Cocina recibe una sola comanda consolidada. Caja cobra una sola cuenta. No se duplican pedidos. No se solapan. El usuario no siente nada."

Esto es alcanzable con los mecanismos propuestos. El código base existe (sync runtime, comandas, transaction_id). Solo falta agregar la capa de detección de conflictos y merge.

Prioridad de implementación
1. 🔴 Detección de mesa ocupada + aviso (1-2 días)
2. 🔴 Lock de mesa con expiración (1 día)
3. 🟡 Merge de comandas (2 días)
4. 🟡 Cobro unificado (1 día)
5. 🟢 Auditoría de conflictos (0.5 día)